

Cloud Services – Week 5

Iago Prol González

Part A – Questions

1. Explain the difference between SAML and OAuth. When would a company choose one over the other?

SAML and OAuth have different purposes. SAML is mainly an authentication protocol, which means that it is used to verify who a user is. It is commonly used for Single Sign-On (SSO) in companies. For example, an employee can log in once with the company identity provider and then access different corporate applications without entering the password again.

OAuth, on the other hand, is mainly an authorization protocol. Instead of proving the identity of the user, it allows an application to access specific resources from another service without giving the application the user's password. For example, an application could be authorized to access a user's calendar using an access token.

A company could choose SAML when it needs enterprise authentication and SSO between corporate applications. OAuth would be more appropriate when an application needs controlled access to APIs or resources from another service. They can also be used together because authentication and authorization are different problems. For modern applications that also need authentication, OpenID Connect (OIDC) can be used on top of OAuth 2.0.

2. What is a Web Application Firewall (WAF) and what kinds of attacks does it defend against? Where would you place one in the three-container architecture from Week 4 (Route → frontend → backend → db)?

A Web Application Firewall (WAF) is a security system that monitors and filters HTTP traffic between users and a web application. Unlike a traditional network firewall, a WAF focuses specifically on web traffic and can detect and block malicious requests before they reach the application.

A WAF can help protect web applications against attacks such as SQL injection, cross-site scripting (XSS) and file inclusion. It uses rules and signatures to identify potentially malicious HTTP requests.

In the three-container architecture that I developed in Week 4, I would place the WAF in front of the public entry point of the application. The architecture would therefore be:

Internet → WAF → Route → Frontend → Backend → MySQL

This is useful because only the frontend is publicly accessible in my Week 4 application, while the backend and MySQL database remain internal. The WAF could inspect incoming web traffic before it reaches the application.

3. What is YAML and why is it the default configuration format for Kubernetes/Rahti manifests? Compare it briefly with JSON.

YAML is a human-readable data serialization language that is commonly used for configuration files. It uses indentation to represent the structure of the data, which makes configuration files relatively easy for people to read and edit.

YAML is commonly used for Kubernetes and Rahti manifests because it is suitable for describing structured configuration in a readable way. For example, in Week 4 I used YAML files to define Deployments, Services, a ConfigMap, a Secret and persistent storage for my application. This allowed the desired configuration of the application to be stored as files and applied to Rahti.

YAML and JSON can represent similar structured data, but their syntax is different. JSON normally uses braces, brackets, commas and quotation marks, while YAML mainly uses indentation and key-value pairs. YAML also supports comments, which is useful in configuration files. JSON is usually more compact for data exchange and can be faster to parse, while YAML is often easier for humans to read and maintain.

4. List some public APIs and describe a realistic integration you could add to the backend built in Week 4.

Public APIs allow applications to obtain data or functionality from external services. There are many different examples, such as weather APIs, transportation APIs, open-data services, financial APIs and geocoding APIs. The Public APIs repository provided in the course contains APIs from many different categories and also indicates whether authentication or an API key is required.

A realistic integration for my Week 4 application would be to add weather information. The Flask backend could make a request to a public weather API and obtain current weather data. The backend would then return this information through its REST API, and the frontend could display values such as temperature, wind speed and weather conditions.

In this architecture, the browser would not need to communicate directly with the external API. Instead, the data flow would be:

Frontend → Flask backend → Public Weather API

The existing MySQL database could continue storing the visit counter from Week 4. This would extend the application with external real-time data while keeping the existing frontend, backend and database architecture.

References:

- Okta. *What are SAML, OAuth, and OIDC?* Updated August 27, 2024.
<https://www.okta.com/identity-101/saml-vs-oauth/>
- Wikipedia contributors. *Web application firewall*. Wikipedia.
https://en.wikipedia.org/wiki/Web_application_firewall
- Wikipedia contributors. *YAML*. Wikipedia.
<https://en.wikipedia.org/wiki/YAML>
- Cruz, M. *Public APIs – A collaborative list of public APIs for developers*. GitHub.
<https://github.com/marcelscruz/public-apis>